

AIKernel Semantic DSL Compiler and Deterministic Agent Execution Architecture

AI生成計画を Semantic IR、ReplayLog、Fail-Closed 制御により Governed Pipeline へコンパイルする実行アーキテクチャ

Author: Takuya Sogawa

Affiliation: AIKernel Project

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: <https://doi.org/10.5281/zenodo.20534341>

Version: v0.1.0-rev1

Date: 2026-06-04

Status: Technical Note / Architecture Draft

License: CC BY 4.0

英語版が正本であり、本稿は companion translation としての日本語版である。

概要

本稿は、LLM が生成する確率的な計画を直接実行せず、OS レベルの決定論的コンパイル境界を通して統治する **AIKernel Semantic DSL Compiler** を提案する。中核となる新規性は、AI が生成する計画をプログラムではなく構造化された意図として扱い、AIKernel がそれを検証・正準化・コンパイル・実行・記録する点にある。

提案する処理経路は、**Semantic IR -> Admissibility Checking -> Governed Pipeline -> ResultStep / SemanticDelta -> hash-linked ReplayLog** である。本モデルにおける DSL は実行可能コードではなく、チューリング完全でもない。これは、Kernel が Admit / Reject / Suspend / Clarify / Compile できる意図レベルの中間表現である。

本アーキテクチャは、自律型エージェントフレームワークでしばしば見られる、計画と実行が単一の非決定論的ループに閉じ込められる問題を扱う。LLM は提案し、Semantic Compiler は Capability 解決、Policy 検査、bounded control-flow 検証を行い、決定論的なパイプラインを構築する。Executor は明示的な SemanticDelta のみを適用し、すべてのステップを ReplayLog のハッシュチェーンに記録する。

本稿は、この Semantic DSL Compiler を AIKernel Phase-2.5 の機構として位置づける。これは Phase-2 の意味論的コンパイル理論と、Phase-3 の JIT Semantic Compiler / 実行時 Semantic OS を接続する橋渡しである。本稿は LLM の計画生成が決定論的になるとは主張しない。非決定論的な提案を、決定論的・監査可能・Fail-Closed な実行成果物へ変換する境界を定義する。

キーワード

AIKernel; Semantic DSL; Semantic IR; Deterministic Agent Execution; Governed Pipeline; ReplayLog; Fail-Closed Governance; Capability Resolution; Semantic Compiler; C#; LINQ; Result Monad; Autonomous AI Agents; Phase-2.5

1. Introduction

LLM ベースのエージェントは、自然言語指示から複雑な多段階タスク計画を生成できる。ツール呼び出し、ファイル検索、リポジトリ参照、文書生成、コード生成、ワークフロー調整などを行えるが、この能力は新しいソフトウェア工学上の問題を生む。多くのシステムでは、エージェントの推論と実行境界が単一の非決定論的ループに畳み込まれている。

既存のエージェントフレームワークでは、モデルがアクションを提案し、ランタイムが局所的な検証だけでそれを実行することが多い。ログが残っても、意味論的な状態遷移が決定論的・再生可能・ポリシー検査済

みの成果物として表現されるとは限らない。そのため、どの Capability がそのツールを許可したのか、どの Policy がステップを admit したのか、どの状態が変化したのか、同じ状態を記録から復元できるのか、といった問いに答えにくい。

AIKernel は、LLM を決定論的 Semantic OS の上で動く確率的プロセスとして扱う。モデルはテキスト、計画、候補、説明を生成してよい。しかし、特権的なシステム状態を直接変更してはならない。確率的な計画と実行の間に、AIKernel は Kernel Boundary として **Semantic DSL Compiler** を置く。

中心となる考え方は単純である。AI が生成した計画は **code ではなく data** として扱うべきである。DSL は構造化された意図宣言である。それは AST に parse され、利用可能な Capability と Governance Rule に対して検証され、制約付き Semantic IR に正準化され、**Governed Pipeline** へコンパイルされる。実行可能なのは、コンパイル済みパイプラインだけである。

2. AIKernel における Phase-2.5 の位置づけ

本稿は **AIKernel Phase-2.5** に位置づけられる。Phase-2 の Semantic Compilation 概念と、ライブ Capability、VFS 状態、ROM Identity、Execution Policy に対して JIT 的に意味論コンパイルを行う将来の Phase-3 Runtime Model をつなぐ。

Phase-1: Capability, VFS, ROM, ReplayLog, Governance

Phase-2: Semantic IR and semantic compilation theory

Phase-2.5: Semantic DSL Compiler for governed agents

Phase-3: JIT Semantic Compiler and runtime semantic OS

Semantic DSL Compiler は、CapabilityROM、VFS、ReplayLog、Fail-Closed Policy Enforcement といった Phase-1 の概念に依存する。同時に、生成された意図を不信任コードとして実行するのではなく、決定論的な実行成果物へコンパイルすることで、Phase-3 の JIT Semantic Compiler に接続する。

3. Background: AIKernel Semantic Runtime

AIKernel は、LLM 中心システムを統治するための Semantic Context OS アーキテクチャである。目的は言語モデルを置き換えることなく、その周囲に決定論的境界を定義することである。

3.1 Provider and Capability

Provider は、モデルエンドポイント、ファイルシステムアクセス、リポジトリ操作、文書変換、検索アダプタ、ドメイン固有 Operator などの能力実装である。**Capability** は、何が使えるか、どの Policy の下で使えるか、どの運用リスクを持つかを記述する。

Semantic DSL Compiler は、任意のツール名がそのまま実行可能な呼び出しになることを許さない。すべての要求は登録済み Capability Table に対して解決される。未知の Provider、欠落した Capability、曖昧な Alias、Policy により拒否された操作は、すべて compile failure となる。

3.2 ReplayLog

ReplayLog は決定論的な証跡記録である。AIKernel では、入力状態、選択された Provider、正準化済みパラメータ、実行出力、Governance Decision、SemanticDelta を記録する。Replay Entry はハッシュチェーンとして連結され、後からの履歴改ざんを検出可能にする。

3.3 Fail-Closed Governance

Fail-Closed Governance とは、未定義、曖昧、不正形式、未許可、Indeterminate な状態をデフォルトで通過させない原則である。Semantic DSL Compiler は、この Fail-Closed Governance を単一プロンプトランザクションから多段階 Agent Pipeline へ拡張する。

4. Design Goals and Non-Goals

本アーキテクチャの設計目標は四つである。

第一に、**Plan Generation** と **Execution Authorization** を分離する。LLM は Pipeline を提案してよいが、その提案が admissible かどうかは Kernel が決定する。

第二に、生成された不信な構造を決定論的な **Semantic IR** に変換する。Compiler は parse、normalization、validation、capability resolution、policy check を担う。

第三に、すべての実行ステップを replayable にする。各ステップは ResultStep、SemanticDelta、hash-linked ReplayLog を生成する。

第四に、自律エージェント向けの bounded control-flow を提供する。Loop、Suspend/Resume、Human Approval Gate、External Event Wait は、明示的制限を持つ governed node として表現されなければならない。

本モデルは、LLM 推論が決定論的になることを主張しない。また、Prompt Injection や Tool Misuse を単独で完全解決するとは主張しない。Sandbox、Permission、Capability Scope、Policy Evaluation、Provider Isolation は引き続き必要である。本稿の貢献は、決定論的なコンパイル境界と実行境界である。

5. Semantic DSL as Structured Semantic IR

DSL は C#、Python、JavaScript、Shell Script、その他のチューリング完全なソース言語ではない。これは意図レベルの実行ステップを記述する制約付き Semantic IR である。AI が生成するのは **構造化された意図** であり、実行可能プログラムではない。

5.1 Semantic IR の四つのスロット

AIKernel Phase-2 の用語と整合させると、Semantic IR 要素は次のように表せる。

$x = (G, T, C, B)$

G: Goal slot - 目的または Root Objective
T: Tool slot - 要求 Capability または Provider Family
C: Context slot - 入力、VFS参照、ROM事実
B: Boundary slot - Policy, Risk, Loop, Approval 制約

この Tuple は記述的であり、それ自体は実行されない。要求された Capability、Context、Boundary 条件が admissible であることを Kernel Compiler が確認した後にのみ、実行可能な Pipeline へ変換される。

5.2 Pipeline Root

有効な DSL 文書は、単一の Pipeline ルートノードを持つ。Pipeline は、bounded step、依存関係、要求 Capability、入力バインディング、出力バインディング、Governance Metadata を含む。

```
{
  "type": "Pipeline",
  "pipelineId": "example-log-review",
  "goal": "Summarize yesterday's application errors",
  "steps": [
    {
      "id": "s1",
      "type": "CapabilityCall",
      "capability": "vfs.readText",
      "input": { "path": "logs/app-2026-06-03.log" },
      "policy": { "mode": "readOnly" }
    },
    {
      "id": "s2",
```

```

    "type": "ModelTransform",
    "dependsOn": ["s1"],
    "capability": "model.summarize",
    "input": { "fromStep": "s1" }
  }
]
}

```

この構造は、コンパイルされるまで実行可能ではない。コンパイル前の時点では単なる提案である。

5.3 Intent Declaration, Not Arbitrary Code

DSL は宣言的である。任意 Loop、eval、Reflection、Dynamic Import、Shell Expansion、無制限 File Path、Open-Ended Tool Execution は公開しない。すべての操作は、既知の Node Type と登録済み Capability に対応しなければならない。

この区別により攻撃面は小さくなる。悪意ある Prompt が危険な Plan を生成させたとしても、その Plan は実行前に Schema Validation、Capability Resolution、Policy Check、Risk Evaluation を通過しなければならない。

6. Semantic Compiler Architecture

Semantic Compiler は提案された DSL 文書を実行可能な Governed Pipeline へ変換する。この経路は四段階である。

6.1 Parsing

Parser は JSON または AST 表現を内部の PipelineNode 構造へ写像する。Parsing は構文的処理であり、Object Shape、Required Field、Type Tag、Identifier を検証する。

6.2 Normalization

Normalization は Identifier を正準化し、Dependency を順序づけ、Alias を解決し、Default を展開し、決定論的表現を生成する。同一の Compiler Version と Policy Context の下では、意味的に等価な Plan は同じ Normalized IR を生成するべきである。

6.3 Formal Admissibility Checking

正準化された Semantic IR 要素を x とする。AIKernel は次の述語が成立する場合にのみ x を admit する。

```

A(x) =
  schema_ok(x)
  AND capability_ok(x)
  AND policy_ok(x)
  AND dataflow_ok(x)
  AND loop_bounded(x)
  AND resource_bounded(x)
  AND provider_bound(x)
  AND canonicalizable(x)
  AND NOT indeterminate(x)

```

受理規則は次である。

```

Accept(x) iff A(x) = true and PDP(x) = Permit
Reject(x) otherwise

```

Admissibility の結果は Admit、Deny、SuspendForApproval、Clarify、Indeterminate のいずれかである。Indeterminate は実行境界では Deny として扱われる。これにより Fail-Closed 契約が維持される。

Fail-Closed rejection は、未知の Node Type、必須 Field 欠落、重複 Step ID、未解決依存、DAG 領域内 Cycle、Capability 欠落、Policy Denial、無制限 Loop、Resume 条件のない外部待機、曖昧な Policy State、Provider Identity の正準化失敗、Replay Hash 入力の決定論的シリアライズ失敗などで発動する。

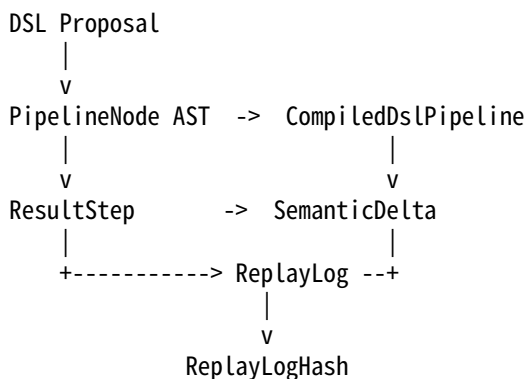
6.4 Pipeline Construction

すべての検査に通過した後、Compiler は Governed Pipeline を構築する。非循環領域は DAG として扱える。Loop 的な制御 Node が存在する場合でも、それは無制限 Runtime Loop ではなく bounded state machine として表現されなければならない。

7. Deterministic Execution Model

コンパイル済み Pipeline は決定論的 Step Evaluation によって実行される。各 Step は Context Snapshot を消費し、ResultStep を返す。

7.1 実行トレース概略



このモデルは意図的に段階化されている。DSL parsing、compilation、execution、delta application、replay hashing は異なる責務である。

7.2 ResultStep and SemanticDelta

ResultStep は、単一の governed step の結果を表す。Step Identifier、Parent Identifier、Normalized Input Hash、Provider Identity、Policy Decision、Output Hash、SemanticDelta を含む。

SemanticDelta は無制御な副作用ではない。これは意図された状態変化の構造的記述である。Executor は Kernel-governed transition rule を通じてのみ Delta を適用する。

$\text{Context}_t + \text{ResultStep}_t + \text{SemanticDelta}_t \rightarrow \text{Context}_{t+1}$

7.3 Hash-Linked ReplayLog

各 Step は Replay Entry を追記する。Entry には直前 Replay Hash が含まれ、決定論的ハッシュチェーンを形成する。

$\text{ReplayHash}_t = H(\text{ReplayHash}_{t-1} \parallel \text{CanonicalReplayEntry}_t)$

ハッシュチェーンは、Plan、Context、Capability Choice、Policy Decision、Step Output、SemanticDelta を束縛する。後からの変更は Replay Hash を変化させ、Trace Verification を破壊する。

7.4 Result Monad and LINQ Bind

C#/NET では、この実行モデルは $\text{Task}\langle\text{Result}\langle T \rangle\rangle$ と LINQ `SelectMany` composition に自然に対応する。想定された Governance Outcome に例外を投げるのではなく、各段階が Result を返す。

```
Task<Result<PipelineContext>> executed =
  from ast in parser.ParseAsync(document)
  from compiled in compiler.CompileAsync(ast, policyContext)
  from result in executor.ExecuteAsync(compiled)
  select result;
```

この形式により、Capability Denial、Malformed Node、Indeterminate Policy Decision は `Result.Failure` として伝播し、決定論的制御フローと Replayability を維持する。

8. Agent Control Flow in DSL

自律型 Agent は直線的 Pipeline だけでは不十分である。Loop、Approval、Wait、Retry、Suspend、Recovery が必要となる。ただし Semantic DSL では、これらはすべて governed control node としてのみ許可される。

8.1 Bounded Loops

Loop と LoopUntil は、maxIterations、timeout、exit condition を必須とする。Compiler は unbounded loop を拒否する。

Loop は、生成コードが制御する任意の while-loop ではなく、bounded state transition region として表現される。各 iteration は replayable step として記録される。

```
{
  "stepId": "loop.check-errors#3",
  "loopId": "loop.check-errors",
  "iteration": 3,
  "maxIterations": 5,
  "exitConditionHash": "...",
  "previousReplayHash": "..."
}
```

Iteration Counter は Hash Input の一部である。したがって Loop Replay は有限であり、因果的に観測可能である。

8.2 Suspend and Resume

一部の操作は、人間による承認や外部イベントを必要とする。その場合、Pipeline は Suspend 状態を発行する。Suspend は成功でも失敗でもない。SemanticDelta を適用しない governed interruption point である。

ReplayLog は Suspension Point、Required Approval、Resume Condition、Current Replay Hash を記録する。Resume 時には、AIKernel は ReplayLog から前回 Context を復元し、Hash Chain を検証し、Approval または Event が Resume Contract を満たすか確認してから、記録された Node から続行する。

Resume には、前回の Suspension Replay Hash が記録と一致すること、Approval または Event が宣言された Resume Contract を満たすこと、の二条件が必要である。どちらかが Indeterminate であれば、Pipeline は Suspend 継続または Fail-Closed となる。

8.3 Retry and Recovery

Retry は bounded recovery policy として宣言されなければならない。上限のない Retry は無効である。Recovery Branch は failed step、admissible error class、maximum retry count を明示する必要がある。

9. Governance and Safety

Semantic DSL Compiler は三層で Governance を適用する。

9.1 Compile-Time Governance

Compile-Time Governance は Provider が呼ばれる前に不正構造を拒否する。Schema、Capability、Dependency、Policy、Loop、Resource Bound を検査する。

9.2 Step-Time Governance

各 Step 実行前に Provider Binding と Context を再検証する。これにより、長時間 Pipeline が Approval 後に Resume した場合でも TOCTOU 的な Drift を防ぐ。

9.3 Post-Step Governance

Step 完了後、SemanticDelta は適用前に検査される。Delta が Policy を超過し、Capability Scope に違反し、Root Goal と衝突する場合、Pipeline は Fail-Closed で停止する。

10. Implementation Profile in AIKernel.NET

参照実装は以下の Interface を中心に構成できる。

```
public interface IDslParser
{
    Task<Result<PipelineNode>> ParseAsync(string document);
}
```

```
public interface ISemanticCompiler
{
    Task<Result<GovernedPipeline>> CompileAsync(
        PipelineNode node,
        PolicyContext policyContext);
}
```

```
public interface IPipelineExecutor
{
    Task<Result<PipelineExecutionResult>> ExecuteAsync(
        GovernedPipeline pipeline,
        KernelExecutionContext context);
}
```

```
public interface IKernelReplayer
{
    Task<Result<KernelExecutionContext>> RestoreAsync(
        ReplayTrace trace);
}
```

実装は AIKernel の性能設計規律に従うべきである。Hot Path で不要な materialization を避け、Replay Hash が byte-level stability に依存する箇所では決定論的 canonicalization を行い、Provider Output は検証されるまで不信として扱う。

11. Evaluation and Validation Matrix

本稿は Architecture Note であり、throughput benchmark は主張しない。以下は参照 Prototype に対する実験的受け入れ条件である。

Test	Procedure	Expected result
Replay determinism	Same DSL, same inputs, same policy, same provider outputs	Identical final ReplayLogHash

Test	Procedure	Expected result
ROM tamper rejection	Modify a CapabilityROM or VFS identity hash before compile	Compilation rejects the pipeline
Capability denial	Request a capability outside policy scope	Result.Failure with no delta applied
Suspend/resume continuity	Suspend for approval, then resume from recorded hash	Resume only if prior hash matches
Bounded loop check	Run <code>LoopUntil</code> with <code>maxIterations = 5</code>	At most five iteration records
Indeterminate PDP	Force policy lookup timeout or ambiguous decision	Fail-closed rejection
Canonicalization stability	Reorder equivalent fields in DSL input	Same normalized IR hash

中心的な実験上の主張は、すべてのタスクが成功することではない。admit されたすべての実行経路が、同一入力、同一 Policy Context、同一 Provider Identity、記録済み Provider Output の下で、governed、replayable、reproducible であることである。

12. Related Work

既存システムは Agent Execution 問題の重要な部分を提供するが、決定論的 Semantic Compilation、Capability-governed admissibility、cryptographic replay trace を一体化しているとは限らない。

System	Relevant strength	Difference from AIKernel
LangChain / LangGraph	Agent orchestration and graph workflows	Logs exist, but semantic deltas and replay hashes are not the kernel boundary
AutoGen	Multi-agent conversation and task coordination	Execution semantics are conversational and less deterministic
Semantic Kernel	Plugins, planners, and process abstractions	It does not define a non-Turing-complete Semantic DSL compiler with ReplayLog hash chains
WASM runtimes	Sandboxed deterministic bytecode execution	They execute code, not semantic intent; governance is not semantic by default
Workflow engines	Explicit process graphs	They usually lack LLM-origin admissibility checks and AIKernel-style capability governance

AIKernel は、Agent Plan を不信な Semantic Artifact として扱い、実行前に決定論的 Kernel Boundary を通して compile する点に重点を置く。主要な対象は単なる Workflow Graph ではなく、hash-linked evidence、explicit capability resolution、fail-closed governance を備えた replayable state-transition contract である。

13. Limitations

本モデルには主に四つの限界がある。

13.1 Expressiveness Limits

DSL は意図的にチューリング完全ではない。これは安全性を高めるが、表現力を制限する。正当な workflow であっても、より小さな governed node へ分解するか、Provider 側実装に移す必要がある。

13.2 Dependence on Provider Determinism

Deterministic Replay は、Provider Behavior が決定論的であるか、Provider Output が記録されていることに依存する。外部 API、時刻依存システム、非決定論的 Model Call は、隔離、記録、Mock、Replay Substitute の対象となる。

13.3 Hash-Chain Stability

Replay Hash の安定性は canonicalization に依存する。Serialization Order、Provider Identity、Policy Version、Timestamp、File Identity、Context Snapshot は、決定論的 Byte Representation として記録されなければならない。

13.4 Intent Extraction Uncertainty

Compiler は構造化された意図を reject または admit できるが、LLM がユーザーの真の意図を完全に抽出したことは保証できない。Clarification、User Confirmation、Root-Goal Governance は引き続き必要である。

14. Conclusion

本稿は、自律型 Agent のための決定論的実行境界として、AIKernel Semantic DSL Compiler を提案した。本モデルは、AI が生成した Plan を実行可能 Code ではなく、構造化 Semantic IR として扱う。それは Capability Resolution、Policy Checking、Control-Flow Bounding、Fail-Closed Validation を経て Governed Pipeline にコンパイルされる。

Execution は明示的な ResultStep と SemanticDelta を通じて進行し、Replay Entry は決定論的ハッシュチェーンとして連結される。これにより、確率的な Agent Proposal は監査可能な Runtime Artifact へ変換される。同時に、「LLM proposes, Kernel governs」という原則が維持される。

Phase-2.5 の機構として、Semantic DSL Compiler は AIKernel の VFS、CapabilityROM、ReplayLog、Governance Layer を、将来の Phase-3 JIT Semantic Compiler へ接続する。今後の課題は、正式な Schema 定義、Replay Hash Canonicalization Test、Recovery Branch Semantics、Distributed Pipeline Execution、より広い AIKernel Governance Layer との統合である。

References

Microsoft. (2026). Semantic Kernel documentation.
<https://learn.microsoft.com/en-us/semantic-kernel/>

Microsoft. (2026). Microsoft Semantic Kernel repository.
<https://github.com/microsoft/semantic-kernel>

LangChain. (2026). LangChain documentation.
<https://docs.langchain.com/>

LangChain. (2026). LangGraph repository: Build resilient agents.
<https://github.com/langchain-ai/langgraph>

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, S., Zhu, E., Jiang, B., Zhang, L., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. OpenReview.
<https://openreview.net/forum?id=BAakY1hNKS>

Moggi, E. (1991). Notions of computation and monads. Information and Computation, 93(1), 55-92.
[https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4)

Wadler, P. (1995). Monads for functional programming. Advanced Functional Programming, 24-52.
https://doi.org/10.1007/3-540-59451-5_2

Sogawa, T. (2026). Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model. Zenodo.
<https://doi.org/10.5281/zenodo.20290614>

Sogawa, T. (2026). AIKernel Formal Foundations: Contract-Based Semantic Execution for Governed AI Systems. Zenodo.
<https://doi.org/10.5281/zenodo.20460322>

Sogawa, T. (2026). AIKernel Hash-Anchored Trust Layer (HATL): A Hybrid Symmetric Ledger with Hash-Based Public Anchors. Zenodo.
<https://doi.org/10.5281/zenodo.20502685>